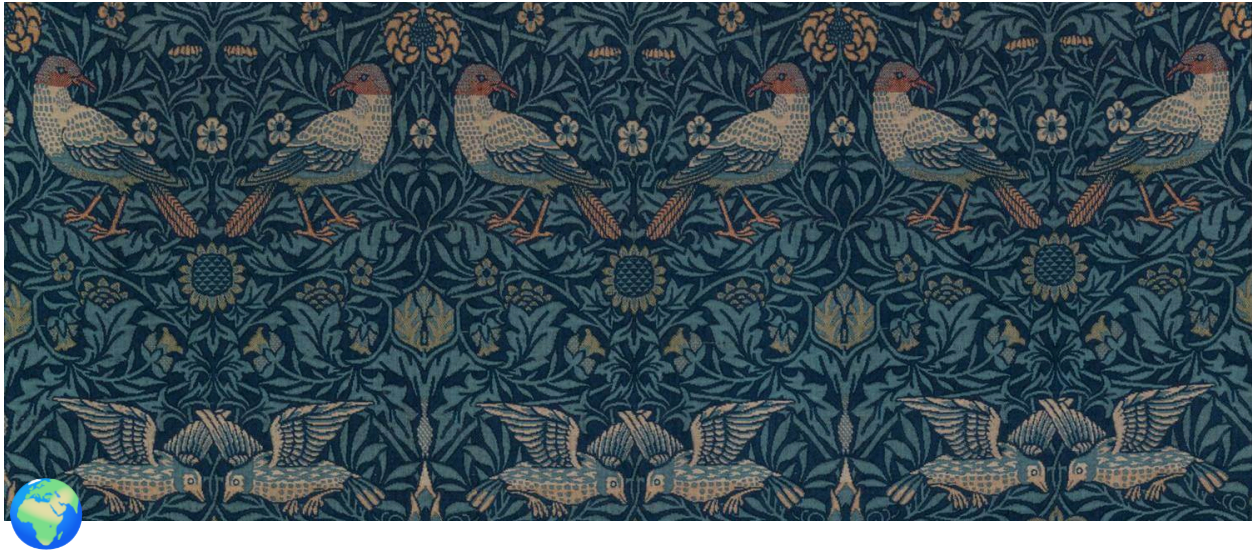




Crusader Colony Sim .b2

▼ sprite engine stub for png's

"""

sprite_engine.py — Layered 8-bit sprite compositing engine for pygame.

Architecture:

- Body layer (16×32) → centered at x+8 on composite
- Weapon layer (32×32) → drawn first (behind body)
- Composite (32×32) → final blit target, scaled at render time

The body metric (Head-Unit system) is baked in:

- 1 HU = 8px | head 0-7 | torso 8-17 | legs 18-31

Usage:

- `sprite = CompositeSprite("hashira", scale=4)`
- `sprite.set_state("attack")`
- `sprite.update(dt)`
- `sprite.draw(screen, (x, y))`

"""

```
import pygame
from enum import Enum, auto
from typing import Optional
```

```
# -----
```

```
# CONSTANTS
```

```
# -----
```

```
BODY_W, BODY_H = 16, 32
```

```

COMP_W, COMP_H = 32, 32
BODY_X_OFFSET = 8 # center 16px body on 32px composite

# HU zone rows (for hitbox generation)

ZONE_HEAD = (0, 7)
ZONE_TORSO = (8, 17)
ZONE_LEGS = (18, 31)

_ = None # transparent shorthand

# -----

# ANIMATION STATE MACHINE

# -----

class SpriteState(Enum):
    IDLE = auto()
    WALK = auto()
    DRAW = auto() # unsheath weapon
    ATTACK = auto()
    SHEATH = auto()
    SPECIAL = auto()
    HIT = auto()

class AnimFrame:
    """One frame of animation – references a body frame index + weapon frame index."""

    __slots__ = ("body_idx", "weapon_idx", "duration_ms")

    def __init__(self, body_idx: int, weapon_idx: int, duration_ms: int = 120):
        self.body_idx = body_idx
        self.weapon_idx = weapon_idx
        self.duration_ms = duration_ms

# -----

# PIXEL DATA → SURFACE HELPERS

# -----

def pixels_to_surface(data: list[list], width: int, height: int) -> pygame.Surface:
    """Convert a 2D list of hex color strings (or None) to a pygame SRCALPHA surface."""
    surf = pygame.Surface((width, height), pygame.SRCALPHA)
    for y, row in enumerate(data):
        for x, color in enumerate(row):
            if color is not None:
                surf.set_at((x, y), pygame.Color(color))
    return surf

def pad_body_to_composite(body_surf: pygame.Surface) -> pygame.Surface:
    """Blit a 16×32 body onto a 32×32 composite (no weapon)."""
    comp = pygame.Surface((COMP_W, COMP_H), pygame.SRCALPHA)

```

```

    comp.blit(body_surf, (BODY_X_OFFSET, 0))
    return comp

def composite_layers(
    body_surf: pygame.Surface,
    weapon_surf: Optional[pygame.Surface],
) -> pygame.Surface:
    """Layer weapon (back) + body (front) onto a 32x32 composite."""
    comp = pygame.Surface((COMP_W, COMP_H), pygame.SRCALPHA)
    if weapon_surf is not None:
        comp.blit(weapon_surf, (0, 0))
    comp.blit(body_surf, (BODY_X_OFFSET, 0))
    return comp

# -----

# HITBOX GENERATOR FROM HU ZONES

# -----

class Hitbox:
    """Axis-aligned hitbox derived from sprite zones + weapon state."""

    __slots__ = ("rect",)

    def __init__(self, rect: pygame.Rect):
        self.rect = rect

    @staticmethod
    def from_sprite(
        world_x: int,
        world_y: int,
        scale: int,
        has_weapon_active: bool,
    ) -> "Hitbox":
        """Generate a hitbox.

        Unarmed: 16px wide (body only), centered in the 32px composite.
        Armed (attack): full 32px composite width.
        """
        if has_weapon_active:
            w = COMP_W * scale
            x = world_x
        else:
            w = BODY_W * scale
            x = world_x + BODY_X_OFFSET * scale
        h = BODY_H * scale
        y = world_y
        return Hitbox(pygame.Rect(x, y, w, h))

    def zone_rect(self, zone: tuple[int, int], scale: int) -> pygame.Rect:
        """Sub-rect for a specific HU zone (head/torso/legs)."""
        y_start, y_end = zone
        return pygame.Rect(

```

```

        self.rect.x,
        self.rect.y + y_start * scale,
        self.rect.width,
        (y_end - y_start + 1) * scale,
    )

# -----

# COMPOSITE SPRITE CLASS

# -----

class CompositeSprite:
    """
    The main sprite class. Holds body frames, weapon frames, animation
    definitions, and handles compositing + rendering.

    Parameters
    -----
    char_id : str
        Key into CHARACTER_REGISTRY (e.g. "akaza", "yuta").
    scale : int
        Render scale (4 = each game pixel → 4×4 screen pixels).
    facing_right : bool
        Initial facing direction.
    """

    def __init__(self, char_id: str, scale: int = 4, facing_right: bool = True):
        reg = CHARACTER_REGISTRY[char_id]

        # Build body surfaces from pixel data
        self.body_frames: list[pygame.Surface] = [
            pixels_to_surface(frame, BODY_W, BODY_H)
            for frame in reg["body_frames"]
        ]

        # Build weapon surfaces (may be empty for unarmed chars)
        self.weapon_frames: list[Optional[pygame.Surface]] = []
        for frame in reg.get("weapon_frames", []):
            if frame is None:
                self.weapon_frames.append(None)
            else:
                self.weapon_frames.append(
                    pixels_to_surface(frame, COMP_W, COMP_H)
                )

        # Animation definitions: state → list[AnimFrame]
        self.animations: dict[SpriteState, list[AnimFrame]] = reg["animations"]

        # Character metadata
        self.char_id = char_id
        self.char_type = reg.get("type", "unarmed") # "unarmed" | "sword"
        self.scale = scale
        self.facing_right = facing_right

```

```

# State machine
self._state = SpriteState.IDLE
self._frame_idx = 0
self._timer_ms = 0.0
self._looping = True

# Precomposite cache (invalidated on frame change)
self._cached_comp: Optional[pygame.Surface] = None
self._cached_scaled: Optional[pygame.Surface] = None
self._cache_key: Optional[tuple] = None

# --- state machine ---

@property
def state(self) -> SpriteState:
    return self._state

def set_state(self, state: SpriteState | str, loop: bool = True):
    """Transition to a new animation state."""
    if isinstance(state, str):
        state = SpriteState[state.upper()]
    if state == self._state:
        return
    if state not in self.animations:
        return # silently ignore states this character doesn't have
    self._state = state
    self._frame_idx = 0
    self._timer_ms = 0.0
    self._looping = loop
    self._invalidate_cache()

@property
def current_anim(self) -> list[AnimFrame]:
    return self.animations.get(self._state, self.animations[SpriteState.IDLE])

@property
def current_frame(self) -> AnimFrame:
    frames = self.current_anim
    return frames[self._frame_idx % len(frames)]

@property
def weapon_active(self) -> bool:
    """True if the current frame has a visible weapon."""
    cf = self.current_frame
    return (
        cf.weapon_idx >= 0
        and cf.weapon_idx < len(self.weapon_frames)
        and self.weapon_frames[cf.weapon_idx] is not None
    )

# --- update / render ---

def update(self, dt_ms: float):

```

```

        """Advance animation by dt_ms milliseconds."""
        frames = self.current_anim
        if len(frames) <= 1:
            return

        self._timer_ms += dt_ms
        cf = frames[self._frame_idx % len(frames)]

        while self._timer_ms >= cf.duration_ms:
            self._timer_ms -= cf.duration_ms
            self._frame_idx += 1

            if self._frame_idx >= len(frames):
                if self._looping:
                    self._frame_idx = 0
                else:
                    self._frame_idx = len(frames) - 1
                    self._timer_ms = 0
                    # Auto-return to idle after non-looping anims
                    if self._state in (SpriteState.ATTACK, SpriteState.DRAW,
                                        SpriteState.SHEATH, SpriteState.HIT):
                        self.set_state(SpriteState.IDLE)
                return

            cf = frames[self._frame_idx % len(frames)]
            self._invalidate_cache()

def _build_composite(self) -> pygame.Surface:
    """Build the 32x32 composite for the current frame."""
    cf = self.current_frame
    body = self.body_frames[cf.body_idx % len(self.body_frames)]

    weapon = None
    if (cf.weapon_idx >= 0 and cf.weapon_idx < len(self.weapon_frames)):
        weapon = self.weapon_frames[cf.weapon_idx]

    return composite_layers(body, weapon)

def _get_scaled(self) -> pygame.Surface:
    """Get the scaled + direction-flipped surface (cached)."""
    key = (self._state, self._frame_idx, self.facing_right, self.scale)
    if self._cache_key == key and self._cached_scaled is not None:
        return self._cached_scaled

    comp = self._build_composite()

    if not self.facing_right:
        comp = pygame.transform.flip(comp, True, False)

    scaled = pygame.transform.scale(
        comp, (COMP_W * self.scale, COMP_H * self.scale)
    )
    self._cached_scaled = scaled
    self._cache_key = key

```

```

        return scaled

    def _invalidate_cache(self):
        self._cache_key = None
        self._cached_scaled = None

    def draw(self, surface: pygame.Surface, pos: tuple[int, int]):
        """Blit the sprite to the target surface at (x, y)."""
        scaled = self._get_scaled()
        surface.blit(scaled, pos)

    def get_hitbox(self, world_x: int, world_y: int) -> Hitbox:
        """Get the current hitbox in world coordinates."""
        return Hitbox.from_sprite(
            world_x, world_y, self.scale, self.weapon_active
        )

    def flip(self):
        """Toggle facing direction."""
        self.facing_right = not self.facing_right
        self._invalidate_cache()

```

PALETTE DEFINITIONS

Each palette is a flat dict — easy to swap for recolors

```

PALETTE = {
    "akaza": {
        "hair": "#E84080", "skin": "#F5C496", "skin_d": "#D4A86A",
        "tattoo": "#3060D0", "eye": "#F0D040", "sash": "#C03030",
        "outfit": "#1A1A2E", "shoe": "#0D0D1A",
    },
    "mahito": {
        "hair": "#6880A8", "skin": "#E0C8A0", "skin_d": "#D0B890",
        "patch": "#C8B088", "stitch": "#303048",
        "eye_l": "#304060", "eye_r": "#60A8C0",
        "hoodie": "#404058", "hoodie_d": "#30303E",
        "pants": "#353548", "pants_d": "#282838", "shoe": "#1A1A28",
    },
    "hashira": {
        "hair": "#2A1810", "skin": "#F0CCA0", "skin_d": "#D8B888",
        "haori_a": "#2A6848", "haori_b": "#1A1A1A",
        "hakama": "#282828", "shoe": "#0D0D18",
        "blade": "#B0B8C0", "blade_d": "#8890A0",
    },
}

```

```

"guard": "#C8A040", "hilt": "#6B3020", "hilt_d": "#4A2018",
},
"yuta": {
"hair": "#1A1A28", "skin": "#E8CCA0", "skin_d": "#D0B888",
"eye": "#304060",
"uniform": "#202848", "uniform_d": "#181838",
"pants": "#181830", "shoe": "#0D0D18",
"blade": "#A0A8B0", "blade_d": "#808890",
"cursed": "#6040A0", "cursed_d": "#4A3080",
"guard": "#C0C0C0", "hilt": "#383040",
},
}

```


CHARACTER PIXEL DATA


```

def akaza_body():
p = PALETTE["akaza"]
H,S,SD,T,E,K,O,SH = (p["hair"],p["skin"],p["skin_d"],p["tattoo"],
p["eye"],p["sash"],p["outfit"],p["shoe"])
return [[
[, , ,H,H,H,H,H,H,, , ],
[, , ,H,H,H,H,H,H,H,H,, , ],
[, ,H,H,H,H,H,H,H,H,H,H,, ],
[, ,H,H,S,S,S,S,S,S,H,H,, ],
[, ,H,T,S,E,S,S,E,S,T,H,, ],
[, ,T,S,S,S,S,S,S,T, , , ],
[, , ,S,T,S,S,T,S, , , , ],
[, , , ,S,S,S,S, , , , , ],
[, , , ,S,SD,SD,S, , , , , ],
[, , ,S,T,S,S,S,S,S,T,S,, , ],
[, ,S,S,T,S,T,S,S,T,S,T,S,S,, ],
[, ,S,S,T,S,T,S,S,T,S,T,S,S,, ],
[, , ,S,T,S,T,K,K,T,S,T,S,, , ],
[, , ,S,T,S,K,K,K,K,S,T,S,, , ],
[, , ,S,T,S,K,K,K,K,S,T,S,, , ],
[, , , ,S,S,K,K,K,K,S,S, , , ],
[, , , ,S,K,K,K,K,S, , , , , ],
[, , , ,O,O,O,O,O,O, , , , , ],
[, , , ,O,O,O,O,O,O,O,O, , , , ],
[, , , ,O,O,O, ,O,O,O, , , , , ],
[, , , ,O,O,O, ,O,O,O, , , , , ],
[, , , ,O,O,O, ,O,O,O, , , , , ],
[, , , ,O,O,O, ,O,O,O, , , , , ],
[, , , ,O,O,O, ,O,O,O, , , , , ],
[, , , ,O,O,O, ,O,O,O, , , , , ],

```

```
[,,, ,O,O,O, ,O,O,O, , , , ],
[,,, ,O,O,O, ,O,O,O, , , , ],
[,,, ,O,O,O, ,O,O,O, , , , ],
[,,, ,O,O,O, ,O,O,O, , , , ],
[,,, ,O,O,O, ,O,O,O, , , , ],
[,,, ,SH,SH,SH, ,SH,SH,SH, , , , ],
[,,, ,SH,SH,SH, ,SH,SH,SH, , ,_-],
]]
```


DEMO — run this file directly to see the roster


```
def main():
    pygame.init()
    SCALE = 5
    MARGIN = 24
    char_ids = list(CHARACTER_REGISTRY.keys())
    n = len(char_ids)

    win_w = n * COMP_W * SCALE + (n + 1) * MARGIN
    win_h = COMP_H * SCALE + MARGIN * 4 + 60 # extra room for labels + controls

    screen = pygame.display.set_mode((win_w, win_h))
    pygame.display.set_caption("Sprite Engine — Roster Demo")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont("monospace", 14)

    sprites = [CompositeSprite(cid, scale=SCALE) for cid in char_ids]
    current_state = SpriteState.IDLE

    state_cycle = [SpriteState.IDLE, SpriteState.ATTACK, SpriteState.DRAW,
    SpriteState.SHEATH, SpriteState.SPECIAL, SpriteState.WALK]

    running = True
    while running:
        dt = clock.tick(60)

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_SPACE:
                    # Cycle through states
                    idx = state_cycle.index(current_state) if current_state in state_cycle else 0
                    idx = (idx + 1) % len(state_cycle)
                    current_state = state_cycle[idx]
                for s in sprites:
                    s.set_state(current_state, loop=(current_state in (
                    SpriteState.IDLE, SpriteState.WALK)))
            elif event.key == pygame.K_f:
```

```

for s in sprites:
    s.flip()
elif event.key == pygame.K_ESCAPE:
    running = False

for s in sprites:
    s.update(dt)

# --- draw ---
screen.fill((18, 18, 26))

for i, (sprite, cid) in enumerate(zip(sprites, char_ids)):
    x = MARGIN + i * (COMP_W * SCALE + MARGIN)
    y = MARGIN * 2
    sprite.draw(screen, (x, y))

# Character name
label = font.render(cid, True, (180, 180, 180))
screen.blit(label, (x + (COMP_W * SCALE - label.get_width()) // 2,
y + COMP_H * SCALE + 8))

# Type badge
badge = font.render(sprite.char_type, True, (100, 160, 120))
screen.blit(badge, (x + (COMP_W * SCALE - badge.get_width()) // 2,
y + COMP_H * SCALE + 24))

# HUD
state_text = font.render(
    f"State: {current_state.name} | SPACE=cycle F=flip ESC=quit",
    True, (120, 120, 140)
)
screen.blit(state_text, (MARGIN, win_h - MARGIN - 16))

pygame.display.flip()
pygame.quit()

if name == "main":
    main()

```

Weapon layer rule

Body stays 16x32. Weapon lives on a separate 32x32 layer. Body centers at x+8 on composite.

body layer - 16x32



+

weapon layer - 32x32



=

composite - 32x32



layer	canvas	anchor	pygame surface (example)
body	16x32	(0,0)	SRCALPHA, blit at (0,0)
weapon	32x32	hand pos	SRCALPHA, blit at (0,0)
composite	32x32	center	final blit target

Result: 16x32 body centered at x+8 on the 32x32 composite, weapon on separate layer for flexible positioning.

YUTA OKKOTSU - SPRITE SHEET

16x32 | 2D FIGHTING GAME | PIXEL ART

IDLE (4 frames)



WALK (4 frames)



SWORD READY / DRAW (4 frames)



SLASH COMBO (4 frames)



THRUST / LUNGE (4 frames)



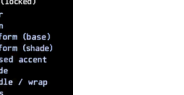
HURT / HIT REACTION (2 frames)



JUMP (2 frames)



CROUCH / GUARD (2 frames)



PALETTE (locked)

- hair
- skin
- uniform (base)
- uniform (shade)
- cursed accent
- blade
- handle / wrap
- eyes
- outlines
- shadow

YUTA
SWORD USER

Full roster - 3x



MERUEM - SPRITE SHEET

16x32 | 2D FIGHTING GAME | PIXEL ART

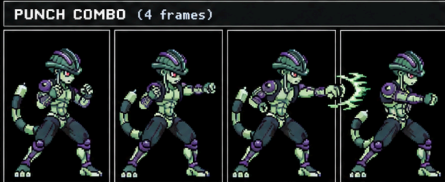
IDLE (4 frames)



WALK (4 frames)



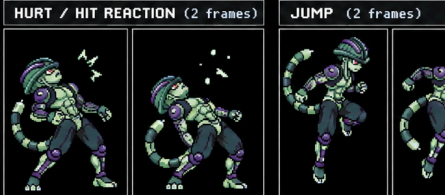
PUNCH COMBO (4 frames)



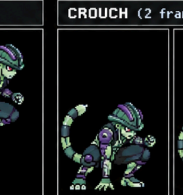
KICK (4 frames)



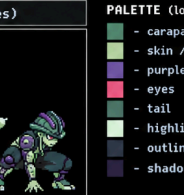
HURT / HIT REACTION (2 frames)



JUMP (2 frames)



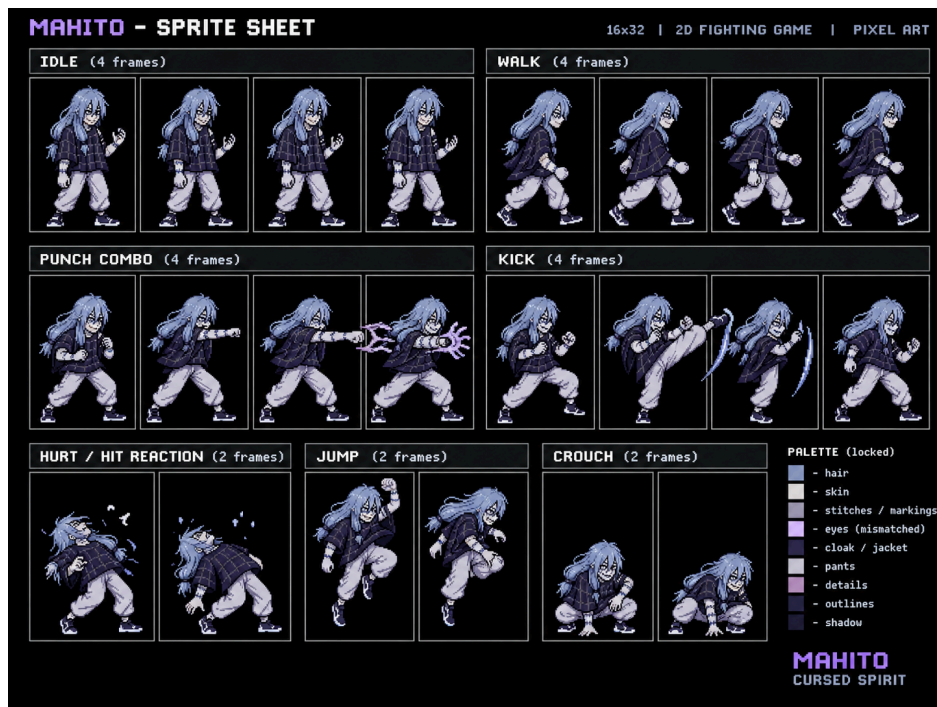
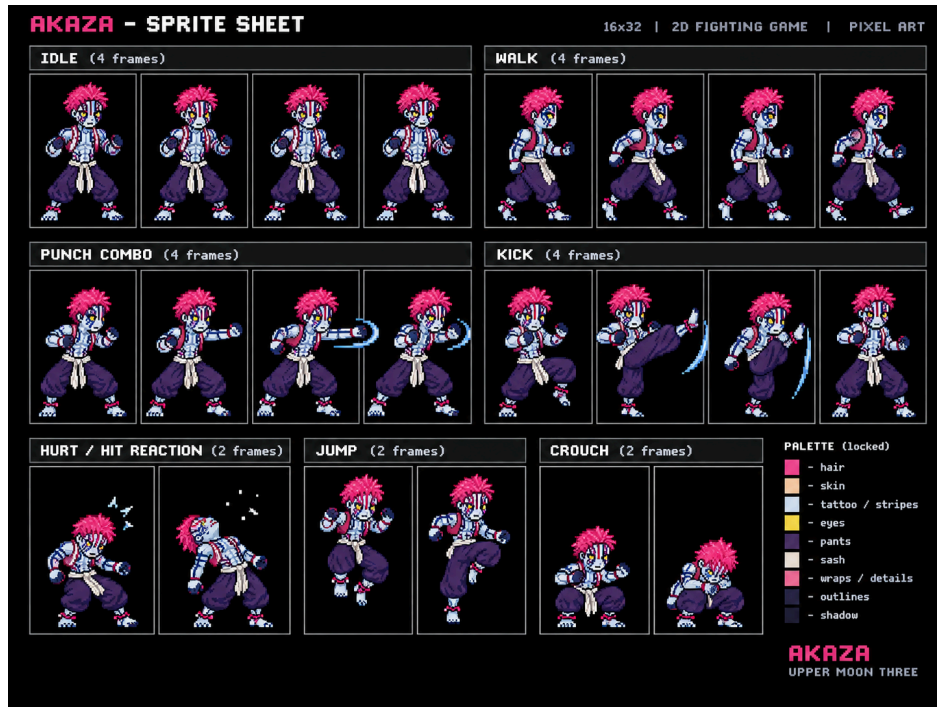
CROUCH (2 frames)



PALETTE (locked)

- carapace green
- skin / shell
- purple armor
- eyes
- tail
- highlights
- outlines
- shadow

MERUEM
CHIMERA ANT KING



"""

sprite_engine_v2.py — Atlas-based sprite compositing engine for pygame.

V2 changes from V1:

- Loads from PNG sprite sheets + JSON manifests (no inline pixel arrays)
- Full animation state set: idle, walk, punch, kick, slash, thrust, hurt, jump, crouch, guard, special
- Three archetypes: unarmed, sword, tail (Meruem)

- Layered compositing: shadow → body → weapon/vfx (z-ordered)
- Per-frame hitbox overrides from manifest
- Placeholder fallback when PNGs aren't present (colored rects)
- Input handler wired to state transitions

Usage:

With real sprite sheets:

```
engine = SpriteEngine()
```

```
engine.load_character("akaza", "assets/akaza_sheet.png", "assets/akaza.json")
```

```
# Or auto-generate placeholders for prototyping:
engine = SpriteEngine()
engine.load_placeholder("akaza", archetype="unarmed", palette=PALETTES["akaza"])

sprite = engine.create_sprite("akaza", scale=4)
sprite.update(dt_ms)
sprite.draw(screen, (x, y))
```

Run directly for the full roster demo:

```
python sprite_engine_v2.py
```

```
"""
```

```
import pygame
```

```
import json
```

```
import os
```

```
from enum import Enum, auto
```

```
from dataclasses import dataclass, field
```

```
from typing import Optional
```

```
from pathlib import Path
```

CONSTANTS & CONFIG

```
BODY_W, BODY_H = 16, 32
```

```
COMP_W, COMP_H = 32, 32
```

```
BODY_X_OFFSET = 8
```

HU metric zones (row ranges)

```
ZONE_HEAD = (0, 7)
```

```
ZONE_TORSO = (8, 17)
```

```
ZONE_LEGS = (18, 31)
```

Standard frame counts per state (from the sprite sheets)

```
STANDARD_FRAME_COUNTS = {
```

```
"idle": 4, "walk": 4,
```

```
"punch": 4, "kick": 4,
```

```
"slash": 4, "thrust": 4,
```

```
"hurt": 2, "jump": 2,
```

```
"crouch": 2, "guard": 2,
```

```
"special": 4,  
}
```

ANIMATION STATES

```
class State(Enum):  
    IDLE = auto()  
    WALK = auto()  
    PUNCH = auto()  
    KICK = auto()  
    SLASH = auto()  
    THRUST = auto()  
    HURT = auto()  
    JUMP = auto()  
    CROUCH = auto()  
    GUARD = auto()  
    SPECIAL = auto()
```

Which states loop vs play-once-then-return-to-idle

```
LOOPING_STATES = {State.IDLE, State.WALK, State.CROUCH, State.GUARD}
```

State name ↔ enum mapping

```
STATE_BY_NAME = {s.name.lower(): s for s in State}
```

CHARACTER ARCHETYPES

```
class Archetype(Enum):  
    UNARMED = "unarmed" # Akaza, Mahito  
    SWORD = "sword" # Yuta, Hashira  
    TAIL = "tail" # Meruem
```

Which attack states each archetype uses

```
ARCHETYPE_ATTACKS = {  
    Archetype.UNARMED: [State.PUNCH, State.KICK],  
    Archetype.SWORD: [State.SLASH, State.THRUST],  
    Archetype.TAIL: [State.PUNCH, State.KICK], # tail composites as weapon layer  
}
```

DATA CLASSES

```
@dataclass
class FrameData:
    """One animation frame's metadata."""
    surface: pygame.Surface
    duration_ms: int = 120
    # Optional per-frame overrides
    hitbox_x_offset: int = 0    # shift hitbox left/right (for lunges)
    hitbox_w_override: int = 0  # 0 = use default
    vfx_surface: Optional[pygame.Surface] = None # slash trail, aura, etc
    vfx_offset: tuple[int, int] = (0, 0)         # blit offset for vfx

@dataclass
class AnimationClip:
    """A sequence of frames for one state."""
    frames: list[FrameData]
    loop: bool = True
    return_to: Optional[State] = None # state to go to after play-once
```

```
@property
def total_duration(self) -> int:
    return sum(f.duration_ms for f in self.frames)
```

```
@dataclass
class CharacterData:
    """Everything needed to instantiate a character sprite."""
    char_id: str
    archetype: Archetype
    animations: dict[State, AnimationClip]
    shadow_surface: Optional[pygame.Surface] = None
    palette: dict = field(default_factory=dict)
```

HITBOX

```
class Hitbox:
    """Axis-aligned hitbox with zone sub-rects."""
```

```
    __slots__ = ("rect",)

    def __init__(self, rect: pygame.Rect):
        self.rect = rect

    def zone(self, zone_rows: tuple[int, int], scale: int) -> pygame.Rect:
        top, bottom = zone_rows
```

```

    return pygame.Rect(
        self.rect.x,
        self.rect.y + top * scale,
        self.rect.width,
        (bottom - top + 1) * scale,
    )

@property
def head(self): return self.zone # call with ZONE_HEAD
@property
def torso(self): return self.zone # call with ZONE_TORSO

def draw_debug(self, surface: pygame.Surface, scale: int):
    """Draw hitbox + zones for debugging."""
    pygame.draw.rect(surface, (255, 255, 0), self.rect, 1)
    pygame.draw.rect(surface, (255, 80, 80),
        self.zone(ZONE_HEAD, scale), 1)
    pygame.draw.rect(surface, (80, 80, 255),
        self.zone(ZONE_TORSO, scale), 1)
    pygame.draw.rect(surface, (80, 255, 80),
        self.zone(ZONE_LEGS, scale), 1)

```

COMPOSITE SPRITE (the runtime object)

class CompositeSprite:

"""

Runtime sprite instance. Handles state machine, frame advancing,
compositing, and rendering.

"""

```

def __init__(self, data: CharacterData, scale: int = 4):
    self.data = data
    self.scale = scale
    self.facing_right = True

    # State machine
    self._state = State.IDLE
    self._frame_idx = 0
    self._timer_ms = 0.0
    self._on_ground = True

    # Rendering cache
    self._cache_key: Optional[tuple] = None
    self._cached_surface: Optional[pygame.Surface] = None

    # World position (set externally or by physics)
    self.x = 0.0
    self.y = 0.0
    self.vel_x = 0.0
    self.vel_y = 0.0

    # — properties —

    @property
    def state(self) -> State:
        return self._state

    @property
    def clip(self) -> AnimationClip:
        if self._state in self.data.animations:
            return self.data.animations[self._state]
        return self.data.animations[State.IDLE]

    @property

```

```

def current_frame(self) -> FrameData:
    frames = self.clip.frames
    return frames[self._frame_idx % len(frames)]

@property
def archetype(self) -> Archetype:
    return self.data.archetype

@property
def is_attacking(self) -> bool:
    return self._state in (State.PUNCH, State.KICK,
                           State.SLASH, State.THRUST, State.SPECIAL)

# — state transitions —

def set_state(self, state: State | str):
    """Transition to a new state. Strings accepted."""
    if isinstance(state, str):
        state = STATE_BY_NAME.get(state.lower(), State.IDLE)
    if state == self._state:
        return
    if state not in self.data.animations:
        return
    # Can't interrupt hurt except with idle
    if self._state == State.HURT and state != State.IDLE:
        return
    self._state = state
    self._frame_idx = 0
    self._timer_ms = 0.0
    self._invalidate()

def _finish_clip(self):
    """Called when a non-looping clip ends."""
    clip = self.clip
    if clip.return_to and clip.return_to in self.data.animations:
        self.set_state(clip.return_to)
    else:
        self.set_state(State.IDLE)

# — update —

def update(self, dt_ms: float):
    """Advance animation + physics."""
    clip = self.clip
    frames = clip.frames
    if not frames:
        return

    self._timer_ms += dt_ms
    cf = frames[self._frame_idx % len(frames)]

    while self._timer_ms >= cf.duration_ms:
        self._timer_ms -= cf.duration_ms
        self._frame_idx += 1

    if self._frame_idx >= len(frames):
        if clip.loop:
            self._frame_idx = 0
        else:
            self._frame_idx = len(frames) - 1
            self._timer_ms = 0
            self._finish_clip()
            return

    cf = frames[self._frame_idx % len(frames)]
    self._invalidate()

# Simple gravity for jump
if not self._on_ground:
    self.vel_y += dt_ms * 0.002 # gravity
    self.y += self.vel_y * dt_ms
    if self.y >= 0:
        self.y = 0
        self.vel_y = 0
        self._on_ground = True
        if self._state == State.JUMP:
            self.set_state(State.IDLE)

```

```

# Walk movement
if self._state == State.WALK:
    speed = 0.15
    dx = speed * dt_ms * (1 if self.facing_right else -1)
    self.x += dx

# — rendering —

def _invalidate(self):
    self._cache_key = None
    self._cached_surface = None

def _render(self) -> pygame.Surface:
    """Build the final scaled surface."""
    key = (self._state, self._frame_idx, self.facing_right, self.scale)
    if self._cache_key == key and self._cached_surface is not None:
        return self._cached_surface

    frame = self.current_frame
    body = frame.surface

    # Determine composite size from body frame
    fw, fh = body.get_size()

    # Build composite: shadow → vfx_behind → body → vfx_front
    comp = pygame.Surface((fw, fh), pygame.SRCALPHA)

    # VFX behind (tail for Meruem)
    if self.archetype == Archetype.TAIL and frame.vfx_surface:
        comp.blit(frame.vfx_surface, frame.vfx_offset)

    # Body
    comp.blit(body, (0, 0))

    # VFX in front (slash trails, aura)
    if self.archetype != Archetype.TAIL and frame.vfx_surface:
        comp.blit(frame.vfx_surface, frame.vfx_offset)

    # Flip
    if not self.facing_right:
        comp = pygame.transform.flip(comp, True, False)

    # Scale
    scaled = pygame.transform.scale(
        comp, (fw * self.scale, fh * self.scale)
    )

    self._cached_surface = scaled
    self._cache_key = key
    return scaled

def draw(self, surface: pygame.Surface, base_pos: tuple[int, int],
        debug_hitbox: bool = False):
    """Draw the sprite. base_pos is the ground-center anchor."""
    rendered = self._render()
    rw, rh = rendered.get_size()

    # Anchor at ground center
    bx, by = base_pos
    draw_x = bx - rw // 2 + int(self.x * self.scale)
    draw_y = by - rh + int(self.y * self.scale)

    # Shadow
    if self.data.shadow_surface:
        sw = self.data.shadow_surface.get_width() * self.scale
        sh = self.data.shadow_surface.get_height() * self.scale
        shadow = pygame.transform.scale(self.data.shadow_surface, (sw, sh))
        shadow.set_alpha(80)
        surface.blit(shadow, (bx - sw // 2, by - sh))

    surface.blit(rendered, (draw_x, draw_y))

    if debug_hitbox:
        hb = self.get_hitbox(draw_x, draw_y)
        hb.draw_debug(surface, self.scale)

def get_hitbox(self, screen_x: int, screen_y: int) -> Hitbox:
    """Get hitbox in screen coordinates."""

```

```

frame = self.current_frame
fw, fh = frame.surface.get_size()

w = fw * self.scale
h = fh * self.scale

# Per-frame override
if frame.hitbox_w_override > 0:
    w = frame.hitbox_w_override * self.scale

x_off = frame.hitbox_x_offset * self.scale
if not self.facing_right:
    x_off = -x_off

return Hitbox(pygame.Rect(screen_x + x_off, screen_y, w, h))

# — input convenience —

def handle_input(self, keys: pygame.key.ScancodeWrapper):
    """Map keyboard to states. Override for custom bindings."""
    if self._state == State.HURT:
        return

    if keys[pygame.K_LEFT]:
        self.facing_right = False
        self.set_state(State.WALK)
        self._invalidate()
    elif keys[pygame.K_RIGHT]:
        self.facing_right = True
        self.set_state(State.WALK)
        self._invalidate()
    elif self._state == State.WALK:
        self.set_state(State.IDLE)

def handle_event(self, event: pygame.event.Event):
    """Handle single key press events for attacks/jumps."""
    if event.type != pygame.KEYDOWN or self._state == State.HURT:
        return

    if event.key == pygame.K_z:
        attacks = ARCHETYPE_ATTACKS[self.archetype]
        self.set_state(attacks[0]) # primary attack
    elif event.key == pygame.K_x:
        attacks = ARCHETYPE_ATTACKS[self.archetype]
        if len(attacks) > 1:
            self.set_state(attacks[1]) # secondary attack
    elif event.key == pygame.K_c:
        self.set_state(State.SPECIAL)
    elif event.key == pygame.K_UP and self._on_ground:
        self.set_state(State.JUMP)
        self._on_ground = False
        self.vel_y = -0.6
    elif event.key == pygame.K_DOWN:
        self.set_state(State.CROUCH)
    elif event.key == pygame.K_LSHIFT:
        self.set_state(State.GUARD)

def flip(self):
    self.facing_right = not self.facing_right
    self._invalidate()

```

SPRITE ENGINE (asset manager + factory)

```

class SpriteEngine:
    """Central registry — loads assets and creates sprite instances."""

```

```

def __init__(self):
    self._registry: dict[str, CharacterData] = {}

# — atlas loading —

def load_character(self, char_id: str, sheet_path: str, manifest_path: str):
    """Load a character from a sprite sheet PNG + JSON manifest.

    Manifest format:
    {
        "archetype": "sword",
        "frame_size": [48, 48],
        "palette": { "hair": "#1A1A28", ... },
        "animations": {
            "idle": {
                "row": 0,
                "frames": 4,
                "duration_ms": 150,
                "loop": true
            },
            "slash": {
                "row": 2,
                "frames": 4,
                "duration_ms": [80, 60, 60, 120],
                "loop": false,
                "vfx_row": 6,
                "vfx_size": [64, 48],
                "vfx_offset": [-8, -8]
            },
            ...
        }
    }
    """
    sheet = pygame.image.load(sheet_path).convert_alpha()
    with open(manifest_path) as f:
        manifest = json.load(f)

    fw, fh = manifest["frame_size"]
    archetype = Archetype(manifest.get("archetype", "unarmed"))
    palette = manifest.get("palette", {})

    animations: dict[State, AnimationClip] = {}

    for state_name, anim_def in manifest["animations"].items():
        state = STATE_BY_NAME.get(state_name)
        if state is None:
            continue

        row = anim_def["row"]
        n_frames = anim_def.get("frames", 4)
        durations = anim_def.get("duration_ms", 120)
        loop = anim_def.get("loop", state in LOOPING_STATES)
        return_to_name = anim_def.get("return_to", None)
        return_to = STATE_BY_NAME.get(return_to_name) if return_to_name else None

        # VFX overlay row (optional)
        vfx_row = anim_def.get("vfx_row", None)
        vfx_size = anim_def.get("vfx_size", [fw, fh])
        vfx_off = tuple(anim_def.get("vfx_offset", [0, 0]))

        if isinstance(durations, int):
            durations = [durations] * n_frames

        frames = []
        for i in range(n_frames):
            # Slice body frame
            rect = pygame.Rect(i * fw, row * fh, fw, fh)
            body_surf = sheet.subsurface(rect).copy()

            # Slice VFX frame if present
            vfx_surf = None
            if vfx_row is not None:
                vw, vh = vfx_size
                vfx_rect = pygame.Rect(i * vw, vfx_row * vh, vw, vh)
                if vfx_rect.right <= sheet.get_width() and \
                    vfx_rect.bottom <= sheet.get_height():
                    vfx_surf = sheet.subsurface(vfx_rect).copy()

```

```

        dur = durations[i] if i < len(durations) else durations[-1]

        # Per-frame hitbox overrides
        hb_overrides = anim_def.get("hitbox_overrides", {})
        hb_frame = hb_overrides.get(str(i), {})

        frames.append(FrameData(
            surface=body_surf,
            duration_ms=dur,
            hitbox_x_offset=hb_frame.get("x_offset", 0),
            hitbox_w_override=hb_frame.get("w_override", 0),
            vfx_surface=vfx_surf,
            vfx_offset=vfx_off,
        ))

    if not loop and return_to is None:
        return_to = State.IDLE

    animations[state] = AnimationClip(
        frames=frames, loop=loop, return_to=return_to
    )

# Shadow surface (small ellipse)
shadow = pygame.Surface((fw, 6), pygame.SRCALPHA)
pygame.draw.ellipse(shadow, (0, 0, 0, 60), (0, 0, fw, 6))

self._registry[char_id] = CharacterData(
    char_id=char_id,
    archetype=archetype,
    animations=animations,
    shadow_surface=shadow,
    palette=palette,
)

# — placeholder generation for prototyping —

def load_placeholder(self, char_id: str, archetype: str = "unarmed",
                    palette: Optional[dict] = None):
    """Generate colored placeholder frames when PNGs aren't ready."""
    if palette is None:
        palette = PALETTES.get(char_id, PALETTES["default"])

    arch = Archetype(archetype)
    frame_w = COMP_W if arch != Archetype.UNARMED else BODY_W
    frame_h = BODY_H

    def make_body_frame(variation: int = 0) -> pygame.Surface:
        surf = pygame.Surface((frame_w, frame_h), pygame.SRCALPHA)
        # Head
        head_c = pygame.Color(palette.get("hair", "#888888"))
        skin_c = pygame.Color(palette.get("skin", "#FFCC99"))
        body_c = pygame.Color(palette.get("outfit", "#444444"))
        leg_c = pygame.Color(palette.get("pants", "#222222"))

        cx = frame_w // 2
        # Head
        pygame.draw.rect(surf, head_c, (cx-4, 0, 8, 4))
        pygame.draw.rect(surf, skin_c, (cx-3, 4, 6, 4))
        # Torso
        pygame.draw.rect(surf, body_c, (cx-5, 8, 10, 10))
        # Arms (shift with variation for walk)
        arm_off = (variation % 2) * 2 - 1
        pygame.draw.rect(surf, skin_c, (cx-7, 9+arm_off, 2, 6))
        pygame.draw.rect(surf, skin_c, (cx+5, 9-arm_off, 2, 6))
        # Legs
        leg_spread = 1 + (variation % 2)
        pygame.draw.rect(surf, leg_c, (cx-4, 18, 3, 12))
        pygame.draw.rect(surf, leg_c, (cx+1, 18, 3, 12))
        # Shoes
        shoe_c = pygame.Color(palette.get("shoe", "#111111"))
        pygame.draw.rect(surf, shoe_c, (cx-4, 30, 3, 2))
        pygame.draw.rect(surf, shoe_c, (cx+1, 30, 3, 2))
        return surf

    def make_weapon_frame(angle_idx: int = 0) -> pygame.Surface:
        surf = pygame.Surface((frame_w, frame_h), pygame.SRCALPHA)
        blade_c = pygame.Color(palette.get("blade", "#B0B8C0"))

```

```

if arch == Archetype.SWORD:
    # Vertical blade for idle, rotate hint for attacks
    if angle_idx == 0:
        pygame.draw.rect(surf, blade_c, (2, 0, 2, 18))
    elif angle_idx == 1:
        pygame.draw.line(surf, blade_c, (4, 16), (28, 10), 2)
    elif angle_idx == 2:
        pygame.draw.line(surf, blade_c, (4, 14), (28, 14), 2)
    else:
        pygame.draw.line(surf, blade_c, (4, 14), (24, 22), 2)
elif arch == Archetype.TAIL:
    tail_c = pygame.Color(palette.get("tail", "#66AA66"))
    # Tail extending from back
    for i in range(10):
        x = frame_w - 4 - i
        y = 20 + (i * i) // 12
        if 0 <= x < frame_w and 0 <= y < frame_h:
            pygame.draw.rect(surf, tail_c, (x, y, 3, 2))
return surf

def make_vfx_frame(state: State, idx: int) -> Optional[pygame.Surface]:
    if state not in (State.SLASH, State.PUNCH, State.KICK, State.SPECIAL):
        return None
    vfx = pygame.Surface((frame_w + 16, frame_h), pygame.SRCALPHA)
    accent = pygame.Color(palette.get("accent", "#8866CC"))
    accent.a = 120
    # Simple arc
    pygame.draw.arc(vfx, accent,
                    (8 + idx * 4, 4, 24, 24),
                    0.5, 2.5, 3)
    return vfx

animations: dict[State, AnimationClip] = {}

# Build all standard states
all_states = {
    State.IDLE: (4, True, 120),
    State.WALK: (4, True, 100),
    State.PUNCH: (4, False, 80),
    State.KICK: (4, False, 80),
    State.SLASH: (4, False, 70),
    State.THRUST: (4, False, 70),
    State.HURT: (2, False, 150),
    State.JUMP: (2, False, 200),
    State.CROUCH: (2, True, 120),
    State.GUARD: (2, True, 120),
    State.SPECIAL: (4, False, 60),
}

for state, (n_frames, loop, dur) in all_states.items():
    frames = []
    for i in range(n_frames):
        body = make_body_frame(variation=i)

        # For sword/tail, composite weapon onto body
        if arch in (Archetype.SWORD, Archetype.TAIL):
            weapon = make_weapon_frame(angle_idx=i if state in (
                State.SLASH, State.THRUST, State.PUNCH,
                State.KICK, State.SPECIAL) else 0)
            comp = pygame.Surface((frame_w, frame_h), pygame.SRCALPHA)
            if arch == Archetype.TAIL:
                comp.blit(weapon, (0, 0))
                comp.blit(body, (0, 0))
            else:
                comp.blit(body, (0, 0))
                comp.blit(weapon, (0, 0))
            body = comp

        vfx = make_vfx_frame(state, i)

        frames.append(FrameData(
            surface=body,
            duration_ms=dur,
            vfx_surface=vfx,
            vfx_offset=(-8, -4) if vfx else (0, 0),
        ))

    animations[state] = AnimationClip(

```

```

        frames=frames,
        loop=loop,
        return_to=State.IDLE if not loop else None,
    )

    # Shadow
    shadow = pygame.Surface((frame_w, 6), pygame.SRCALPHA)
    pygame.draw.ellipse(shadow, (0, 0, 0, 60), (0, 0, frame_w, 6))

    self._registry[char_id] = CharacterData(
        char_id=char_id,
        archetype=arch,
        animations=animations,
        shadow_surface=shadow,
        palette=palette,
    )

def create_sprite(self, char_id: str, scale: int = 4) -> CompositeSprite:
    """Instantiate a sprite from registered character data."""
    if char_id not in self._registry:
        raise KeyError(f"Character '{char_id}' not registered. "
                       f"Available: {list(self._registry.keys())}")
    return CompositeSprite(self._registry[char_id], scale=scale)

@property
def characters(self) -> list[str]:
    return list(self._registry.keys())

```

PALETTES (locked per character, from the sprite sheets)

```

PALETTES = {
    "default": {
        "hair": "#888888", "skin": "#FFCC99", "outfit": "#444444",
        "pants": "#333333", "shoe": "#111111", "accent": "#8866CC",
    },
    "akaza": {
        "hair": "#E84080", "skin": "#C8A0D0", "tattoo": "#6060C0",
        "outfit": "#C8A0D0", "pants": "#2A1A3E", "shoe": "#C03030",
        "sash": "#D8C090", "accent": "#60C0E0",
        "eyes": "#E8D040",
    },
    "mahito": {
        "hair": "#6880A8", "skin": "#E0C8A0", "outfit": "#404058",
        "pants": "#353548", "shoe": "#1A1A28",
        "stitch": "#303048", "accent": "#60A8C0",
    },
    "yuta": {
        "hair": "#1A1A28", "skin": "#E8CCA0", "outfit": "#202848",
        "pants": "#181830", "shoe": "#0D0D18",
        "blade": "#A0A8B0", "accent": "#8060C0",
    },
    "meruem": {
        "hair": "#608848", "skin": "#90B070", "outfit": "#303840",
        "pants": "#283030", "shoe": "#1A2020",
        "armor": "#706098", "tail": "#70A858",
    }
}

```

```

"eyes": "#E05050", "accent": "#60D060",
},
"hashira": {
"hair": "#2A1810", "skin": "#F0CCA0", "outfit": "#2A6848",
"pants": "#282828", "shoe": "#0D0D18",
"blade": "#B0B8C0", "accent": "#60A8E0",
},
}

```

JSON MANIFEST TEMPLATE GENERATOR

```

def generate_manifest_template(char_id: str, archetype: str = "unarmed",
frame_w: int = 48, frame_h: int = 48) -> dict:
    """Generate a starter JSON manifest for a character sheet.

```

```

    The artist lays out frames left-to-right in rows, one row per state.
    This generates the row mapping. Save it and tweak durations/hitboxes.
    """
    attacks = {
        "unarmed": ["punch", "kick"],
        "sword": ["slash", "thrust"],
        "tail": ["punch", "kick"],
    }

    states_order = ["idle", "walk"] + attacks.get(archetype, ["punch", "kick"]) + [
        "hurt", "jump", "crouch", "special"
    ]

    manifest = {
        "archetype": archetype,
        "frame_size": [frame_w, frame_h],
        "palette": PALETTES.get(char_id, PALETTES["default"]),
        "animations": {},
    }

    for row_idx, state_name in enumerate(states_order):
        n_frames = STANDARD_FRAME_COUNTS.get(state_name, 4)
        loop = state_name in ("idle", "walk", "crouch", "guard")
        dur = 120 if loop else 80

        entry = {
            "row": row_idx,
            "frames": n_frames,
            "duration_ms": dur,
            "loop": loop,
        }

        # Add VFX hints for attack states
        if state_name in ("slash", "thrust", "punch", "kick", "special"):
            entry["vfx_row"] = None # artist fills this in
            entry["vfx_size"] = [frame_w + 16, frame_h]
            entry["vfx_offset"] = [-8, 0]
            entry["hitbox_overrides"] = {
                "2": {"x_offset": 4, "w_override": frame_w + 8},
            }

        if not loop:
            entry["return_to"] = "idle"

        manifest["animations"][state_name] = entry

```

```
return manifest
```

```
def save_manifest_template(char_id: str, output_dir: str = "assets",
archetype: str = "unarmed"):
    """Save a JSON manifest template to disk."""
    os.makedirs(output_dir, exist_ok=True)
    manifest = generate_manifest_template(char_id, archetype)
    path = os.path.join(output_dir, f"{char_id}.json")
    with open(path, "w") as f:
        json.dump(manifest, f, indent=2)
    print(f"Saved manifest template: {path}")
    return path
```

DEMO

```
def main():
    pygame.init()
```

```
    SCALE = 5
    GROUND_Y = 360
    WIN_W, WIN_H = 900, 440

    screen = pygame.display.set_mode((WIN_W, WIN_H))
    pygame.display.set_caption("Sprite Engine v2 - Roster Demo")
    clock = pygame.time.Clock()

    try:
        font = pygame.font.SysFont("monospace", 13)
        font_big = pygame.font.SysFont("monospace", 16, bold=True)
    except Exception:
        font = pygame.font.Font(None, 14)
        font_big = pygame.font.Font(None, 18)

    # — load roster —
    engine = SpriteEngine()

    roster = [
        ("akaza", "unarmed"),
        ("mahito", "unarmed"),
        ("yuta", "sword"),
        ("meruem", "tail"),
        ("hashira", "sword"),
    ]

    for char_id, arch in roster:
        # Try loading real assets first
        sheet_path = f"assets/{char_id}_sheet.png"
        manifest_path = f"assets/{char_id}.json"
        if os.path.exists(sheet_path) and os.path.exists(manifest_path):
            engine.load_character(char_id, sheet_path, manifest_path)
        else:
            engine.load_placeholder(char_id, archetype=arch)

    # Create sprites
    char_ids = engine.characters
    sprites = [engine.create_sprite(cid, scale=SCALE) for cid in char_ids]
    selected = 0
    debug_hitbox = False

    # State display
```

```

state_names = [s.name.lower() for s in State]

running = True
while running:
    dt = clock.tick(60)

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_ESCAPE:
                running = False
            elif event.key == pygame.K_TAB:
                selected = (selected + 1) % len(sprites)
                # Reset position
                for s in sprites:
                    s.x = 0
            elif event.key == pygame.K_h:
                debug_hitbox = not debug_hitbox
            elif event.key == pygame.K_g:
                # Generate manifest templates
                for char_id, arch in roster:
                    save_manifest_template(char_id, archetype=arch)
        else:
            sprites[selected].handle_event(event)

    # Continuous input for active sprite
    keys = pygame.key.get_pressed()
    sprites[selected].handle_input(keys)

    # Update all
    for s in sprites:
        s.update(dt)

    # — draw —
    screen.fill((14, 14, 22))

    # Ground line
    pygame.draw.line(screen, (40, 40, 50), (0, GROUND_Y), (WIN_W, GROUND_Y))

    # Draw roster
    spacing = WIN_W // (len(sprites) + 1)
    for i, (sprite, cid) in enumerate(zip(sprites, char_ids)):
        bx = spacing * (i + 1)
        by = GROUND_Y

        is_selected = i == selected
        sprite.draw(screen, (bx, by), debug_hitbox=debug_hitbox)

        # Name tag
        color = (230, 230, 240) if is_selected else (100, 100, 110)
        name_surf = (font_big if is_selected else font).render(
            cid, True, color)
        screen.blit(name_surf,
            (bx - name_surf.get_width() // 2, GROUND_Y + 8))

        # Archetype badge
        arch_color = {
            Archetype.UNARMED: (200, 100, 100),
            Archetype.SWORD: (100, 150, 220),
            Archetype.TAIL: (100, 200, 120),
        }[sprite.archetype]
        badge = font.render(sprite.archetype.value, True, arch_color)
        screen.blit(badge,
            (bx - badge.get_width() // 2, GROUND_Y + 24))

        # State
        if is_selected:
            state_surf = font.render(
                sprite.state.name, True, (180, 180, 60))
            screen.blit(state_surf,
                (bx - state_surf.get_width() // 2, GROUND_Y + 38))

        # Selection marker
        if is_selected:
            pygame.draw.polygon(screen, (240, 200, 60), [
                (bx, GROUND_Y + 56), (bx - 5, GROUND_Y + 64),
                (bx + 5, GROUND_Y + 64),

```

```

    })

    # HUD
    hud_lines = [
        "TAB=select ARROWS=move Z=atk1 X=atk2 C=special",
        "UP=jump DOWN=crouch SHIFT=guard H=hitbox G=gen manifests",
        f"FPS: {clock.get_fps():.0f} | selected: {char_ids[selected]} | "
        f"state: {sprites[selected].state.name}",
    ]
    for i, line in enumerate(hud_lines):
        surf = font.render(line, True, (80, 80, 100))
        screen.blit(surf, (12, WIN_H - 50 + i * 15))

    pygame.display.flip()

pygame.quit()

```

```

if name == "main":
    main()

```